



SETUP GUIDE

KUBEFLOW + MLFLOW

AI Platform Engineering (Module 5)

This guide covers installation and verification of:

- MLflow (experiment tracking + model registry)
- Kubeflow Pipelines SDK (pipeline compilation)
- Full Kubeflow on Kubernetes (optional — requires 8+ GB RAM)

IMPORTANT: Complete MLflow setup at minimum. KFP SDK is recommended.
Full Kubeflow on K8s is optional and resource-intensive.

PREREQUISITES

Before starting, ensure you have:

- [] Python 3.8+ installed (python3 --version)
- [] pip installed (pip3 --version)
- [] A virtual environment tool (venv or conda)
- [] Internet connection for package downloads

For Full Kubeflow (optional):

- [] Docker Desktop installed and running (from Week 2 or Week 4 prep)
- [] 8+ GB RAM available (16 GB recommended)
- [] kubectl installed (will be covered in Week 5)
- [] Minikube installed (will be covered in Week 5)

SECTION 1: MLFLOW SETUP

MLflow runs on all platforms — Windows, macOS, Linux. No special setup.

Step 1: Create a Virtual Environment

Navigate to your working directory (MUST be on the Linux filesystem, NOT /mnt/c/)

WSL users: virtual environments on /mnt/c/ will fail with

"externally-managed-environment" errors. Always use ~/

```
mkdir -p ~/platform-engineering
```



```
cd ~/platform-engineering

# Create and activate virtual environment

python3 -m venv mlflow-env

# Activate:

# Linux/macOS/WSL:

source mlflow-env/bin/activate

# Windows PowerShell:

.\mlflow-env\Scripts\Activate.ps1

# Windows CMD:

mlflow-env\Scripts\activate.bat
```

Step 2: Install MLflow

```
pip install mlflow
```

Step 3: Start the MLflow Tracking Server

```
mlflow server --host 0.0.0.0 --port 5000

# Expected output:

# [INFO] Starting gunicorn ...

# [INFO] Listening at: http://0.0.0.0:5000

# Open browser: http://localhost:5000

# You should see the MLflow UI with an empty experiments list.

# Leave this terminal running. Open a NEW terminal for the next steps.
```

Step 4: Run a Test Experiment

```
# In a new terminal, activate the same virtual environment

cd ~/platform-engineering

source mlflow-env/bin/activate # (or Windows equivalent)

# Create test script:

cat << 'EOF' > test_mlflow.py

import mlflow
```

```
# Set tracking URI (local server)

mlflow.set_tracking_uri("http://localhost:5000")

# Create an experiment

mlflow.set_experiment("week3-test")

# Log a run

with mlflow.start_run():

    mlflow.log_param("learning_rate", 0.01)

    mlflow.log_param("epochs", 50)

    mlflow.log_metric("accuracy", 0.94)

    mlflow.log_metric("f1_score", 0.91)

    print("Run logged successfully!")

    print("Check http://localhost:5000 for your experiment.")

EOF

python3 test_mlflow.py
```

Step 5: Verify in the UI

1. Open <http://localhost:5000>
2. Click "week3-test" experiment on the left sidebar
3. You should see one run with:
Parameters: learning_rate=0.01, epochs=50
Metrics: accuracy=0.94, f1_score=0.91
4. Click the run to see full details
5. If you see this → MLflow is working correctly.

Step 6: Test Model Logging

```
import mlflow

import mlflow.sklearn

from sklearn.linear_model import LogisticRegression

from sklearn.datasets import make_classification

import numpy as np
```

```
mlflow.set_tracking_uri("http://localhost:5000")

mlflow.set_experiment("week3-model-test")

# Create dummy data

X, y = make_classification(n_samples=100, n_features=4, random_state=42)

# Train a simple model

model = LogisticRegression()

model.fit(X, y)

accuracy = model.score(X, y)

# Log to MLflow

with mlflow.start_run():

    mlflow.log_param("model_type", "LogisticRegression")

    mlflow.log_metric("accuracy", accuracy)

    mlflow.sklearn.log_model(model, "model")

    print(f"Model logged with accuracy: {accuracy:.4f}")

    print("Check the MLflow UI — click the run to see the model artifact.")

EOF

pip install scikit-learn

python3 test_mlflow_model.py

# In the UI: click the run → Artifacts tab → you should see "model/" folder
```

SECTION 2: KUBEFLOW PIPELINES SDK (KFP)

The KFP SDK lets you define and compile pipelines locally — no Kubernetes cluster needed.

Step 1: Install KFP SDK

In the same virtual environment

```
pip install kfp
```

Step 2: Create a Test Pipeline

```
cat << 'EOF' > test_kfp_pipeline.py
```

```
from kfp import dsl, compiler
```

```
@dsl.component(base_image="python:3.10-slim")
```

```
def say_hello(name: str) -> str:
```

```
    message = f"Hello, {name}!"
```

```
    print(message)
```

```
    return message
```

```
@dsl.component(base_image="python:3.10-slim")
```

```
def process_message(message: str) -> str:
```

```
    result = message.upper()
```

```
    print(result)
```

```
    return result
```

```
@dsl.pipeline(name="test-pipeline", description="Week 3 test pipeline")
```

```
def test_pipeline(name: str = "Platform Engineer"):
```

```
    hello_task = say_hello(name=name)
```

```
    process_task = process_message(message=hello_task.output)
```

```
    # Compile to YAML
```

```
    compiler.Compiler().compile(
```

```
        pipeline_func=test_pipeline,
```

```
        package_path="test_pipeline.yaml"
```

```
)
```

```
    print("Pipeline compiled successfully! Check test_pipeline.yaml")
```

```
EOF
```

```
python3 test_kfp_pipeline.py
```

Step 3: Verify Compilation

Check the YAML was created

```
ls -la test_pipeline.yaml
```

View the structure (should show pipeline spec with two components)

```
head -50 test_pipeline.yaml
```

Expected: YAML with pipelineSpec, components, deploymentSpec

If you see valid YAML → KFP SDK is working correctly.

SECTION 3: FULL KUBEFLOW ON KUBERNETES (OPTIONAL)

⚠ RESOURCE WARNING: Kubeflow requires 8+ GB RAM allocated to Minikube.

If your machine has less than 16 GB total RAM, skip this section and use the KFP SDK (Section 2) for local development.

⚠ PREREQUISITE: This section assumes kubectl and Minikube are installed.

If not, follow the Week 5 Setup Guide first, then return here.

Option A: KFP Standalone on Minikube (Lighter)

This installs only Kubeflow Pipelines, not the full Kubeflow platform.

Start Minikube with sufficient resources

```
minikube start --cpus=4 --memory=8192 --driver=docker
```

Deploy KFP standalone

```
export PIPELINE_VERSION=2.0.5
```

```
kubectl apply -k "github.com/kubeflow/pipelines/manifests/kustomize/cluster-scoped-resources?  
ref=$PIPELINE_VERSION"
```

```
kubectl wait --for condition=established --timeout=60s crd/applications.app.k8s.io
```

```
kubectl apply -k "github.com/kubeflow/pipelines/manifests/kustomize/env/platform-agnostic?  
ref=$PIPELINE_VERSION"
```

Wait for pods to be ready (this takes 5-10 minutes)

```
kubectl -n kubeflow wait --for=condition=Ready pods --all --timeout=600s
```

Port-forward the UI



```
kubectrl port-forward -n kubeflow svc/ml-pipeline-ui 8080:80
```

Open browser: <http://localhost:8080>

You should see the Kubeflow Pipelines UI.

Option B: KFP SDK Only (No Cluster)

If you cannot run Minikube, you can still:

1. Define pipelines with the KFP SDK (Section 2 above)
2. Compile them to YAML
3. Upload the YAML to a shared Kubeflow cluster (if available)
4. Use Google Cloud Vertex AI Pipelines (accepts KFP YAML)

This is sufficient for learning pipeline definition and compilation.

SECTION 4: OS-SPECIFIC NOTES

Windows (WSL)

All commands in this guide run inside WSL (Ubuntu).

1. Open WSL terminal: `wsl` in PowerShell or open Ubuntu from Start menu
2. Python: `sudo apt install python3 python3-pip python3-venv`
3. MLflow server: runs in WSL, accessible from Windows browser at localhost
4. Minikube (optional): install inside WSL, use `--driver=docker`

→ Requires Docker Desktop with WSL 2 backend enabled

Potential issue: "localhost not accessible from Windows browser"

Fix: Use the WSL IP address instead:

```
hostname -I # Shows WSL IP, e.g., 172.x.x.x
```

Use <http://172.x.x.x:5000> in Windows browser

macOS

1. Python: `brew install python3` (or use system Python 3.8+)
2. All pip commands work as shown
3. MLflow server: accessible at localhost:5000
4. Minikube (optional): `brew install minikube`

→ Use `--driver=docker` (requires Docker Desktop)

Linux (Ubuntu/Debian)

1. Python: `sudo apt install python3 python3-pip python3-venv`
2. All commands work as shown
3. Minikube (optional):

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
→ Use --driver=docker (requires Docker installed)
```

SECTION 5: TROUBLESHOOTING

MLflow Issues

Problem: "mlflow: command not found"

Fix: Ensure virtual environment is activated

```
pip install mlflow (re-run)
```

Or use: `python3 -m mlflow server ...`

Problem: "Address already in use" on port 5000

Fix: Kill existing process:

```
lsof -i :5000 # Find PID
```

```
kill <PID>
```

Or use a different port:

```
mlflow server --host 0.0.0.0 --port 5001
```

Problem: macOS port 5000 conflict (AirPlay Receiver)

Fix: System Settings → General → AirDrop & Handoff → AirPlay Receiver OFF

Or use port 5001 instead

Problem: MLflow UI shows no experiments after running test script

Fix: Ensure tracking URI matches:

- Server started with: `mlflow server --host 0.0.0.0 --port 5000`
- Script uses: `mlflow.set_tracking_uri("http://localhost:5000")`
- Both in the same network (WSL: use WSL IP, not localhost)

KFP SDK Issues

Problem: "ModuleNotFoundError: No module named 'kfp'"

Fix: Ensure virtual environment is activated

```
pip install kfp
```

Problem: Compilation error "invalid component"

Fix: Ensure `@dsl.component` has `base_image` parameter

Ensure function signature has type annotations

Check KFP version: `pip show kfp` (should be 2.x)



Minikube/Kubeflow Issues

Problem: "minikube start" fails with insufficient memory

Fix: Allocate more memory: minikube start --memory=8192

Or close other applications to free RAM

Minimum: 8 GB for Minikube, 16 GB total system RAM recommended

Problem: Kubeflow pods stuck in "Pending" or "CrashLoopBackOff"

Fix: Check resources: kubectl top nodes

Check pod logs: kubectl -n kubeflow logs <pod-name>

Increase Minikube resources and restart

Problem: Port-forward disconnects

Fix: Re-run: kubectl port-forward -n kubeflow svc/ml-pipeline-ui 8080:80

This is expected — port-forward is not persistent

SECTION 6: VERIFICATION CHECKLIST

Complete these checks before the live session:

MLflow (Required):

- ☐ Virtual environment created and activated
- ☐ MLflow installed (mlflow --version shows 2.x)
- ☐ MLflow server starts without errors
- ☐ MLflow UI accessible in browser (localhost:5000)
- ☐ Test experiment appears in UI after running test script
- ☐ Model artifact visible in UI after running model test script

KFP SDK (Recommended):

- ☐ kfp installed (pip show kfp shows 2.x)
- ☐ Test pipeline compiles to YAML without errors
- ☐ YAML file contains valid pipeline specification

Full Kubeflow on K8s (Optional):

- ☐ Minikube running with 8+ GB RAM
- ☐ Kubeflow pods all in "Running" state
- ☐ Kubeflow UI accessible at localhost:8080
- ☐ Can upload and run the test pipeline YAML

Cleanup commands (after verification):

Stop MLflow server: Ctrl+C in the terminal running it

Stop Minikube (if started): minikube stop



Deactivate virtual environment: deactivate